

Initiation à l'environnement Unix

CM1 : Historique et arborescence des fichiers

Pierre Rousselin
`rousselin@univ-paris13.fr`

Université Sorbonne Paris Nord
L1 informatique et double licence
septembre 2024

Organisation du cours

Le système Unix

Le shell

Demos 2025

Lignes de commandes

Organisation du cours

Le système Unix

Le shell

Demos 2025

Lignes de commandes

Cours et ressources

- ▶ 1h30 de CM + 3h de TP pendant 11 semaines.
- ▶ Page sur moodle : sujets de TP, transparents du CM, ...
- ▶ Responsable du cours : Pierre Rousselin.
`rousselin@univ-paris13.fr`
- ▶ 10 groupes de TP (2 groupes DL + 8 groupes Info)
- ▶ Pas de cours, pas de TP la semaine du 27 octobre (vacances)
- ▶ Pas de cours, pas de TP la semaine du 8 décembre¹

1. sauf rattrapage éventuel, au cas par cas selon les groupes

Note du module

- ▶ Épreuve écrite (Pa_1) la semaine du 20 octobre (2h)
- ▶ Évaluation continue (EvC) : QCM chaque semaine (et peut-être d'autres rendus)
- ▶ Examen final (Pa_2) la semaine du 15 décembre.
- ▶ Note à la fin du semestre :

$$N_{\text{unix}} = \frac{\text{EvC} + 2Pa_1 + 4Pa_2}{7}$$

Note du module

- ▶ Épreuve écrite (Pa_1) la semaine du 20 octobre (2h)
- ▶ Évaluation continue (EvC) : QCM chaque semaine (et peut-être d'autres rendus)
- ▶ Examen final (Pa_2) la semaine du 15 décembre.
- ▶ Note à la fin du semestre :

$$N_{\text{unix}} = \frac{\text{EvC} + 2Pa_1 + 4Pa_2}{7}$$

- ▶ Si $N_{\text{unix}} < 10$ et passage au niveau 2, alors épreuve de seconde chance (Pa) la semaine du 8 juin et la note du module est

$$N'_{\text{unix}} = \sup \left\{ N_{\text{unix}}, \frac{\text{EvC} + 6Pa}{7}, Pa \right\}$$

Pourquoi ce module ?

- ▶ Utilité immédiate : savoir utiliser les commandes Unix pour obtenir très rapidement ce qu'on veut. Utile dans *tous* les modules avec des TP machines.
- ▶ Utilité à long terme : comprendre (en manipulant) les concepts principaux (processus, fichiers, flux, utilisateur, ...) du merveilleux système d'exploitation Unix. Suite en L2 : module *Système d'exploitation*.
- ▶ Utilité à long terme : devenir autonome sur un système Unix, pouvoir se débrouiller seul et avoir le contrôle de sa machine. Suite en L2 : module *Administration système*.
- ▶ Utilité à long terme : s'imprégner de la « philosophie Unix » qui donne un très bon exemple de concepts de génie logiciel encore utiles aujourd'hui (modularité, simplicité, ...).

Organisation du cours

Le système Unix

Le shell

Demos 2025

Lignes de commandes



Ken Thompson et Dennis Ritchie (assis) travaillant sur un PDP-11
Par Peter Hamer, téléversé par Magnus Manske, CC BY-SA 2.0,
<https://commons.wikimedia.org/w/index.php?curid=24512134>

Système d'exploitation

Unix est un *système d'exploitation* (en anglais *operating system* ou O.S.). Un (noyau de) système d'exploitation est un programme :

- ▶ qui est chargé dans la mémoire vive (RAM) au démarrage de la machine ;
- ▶ qui a le monopole de l'accès au *matériel* :
 - ▶ processeur(s) et mémoire vive
 - ▶ autres composants de calcul (GPU, ...)
 - ▶ périphériques d'entrée : clavier, souris, tablette graphique, microphone, webcam, ...
 - ▶ périphériques de sortie : écran, imprimante, enceintes, ...
 - ▶ connexions réseau (à la fois entrée et sortie) : par ethernet, wifi, ...
- ▶ qui charge les autres programmes en mémoire ;
- ▶ et leur donne la possibilité de lui demander gentiment l'accès à une ressource matérielle : ce sont les *appels système*.

Organisation du système Unix

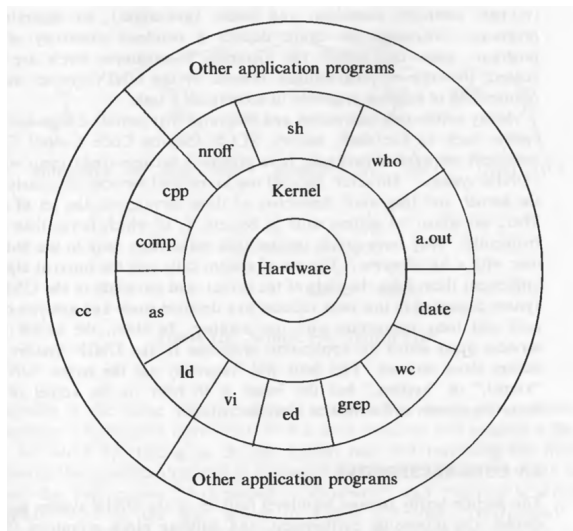


Figure – Source : *Design of the Unix Operating System*, Maurice Bach, 1986

Système Unix

- ▶ Multi-utilisateur
- ▶ L'utilisateur communique avec l'ordinateur par le biais d'un *terminal*.
- ▶ Multitâche (illusion que plusieurs programmes « tournent » en même temps), à *temps partagé* (*time-sharing*)
- ▶ Système de fichiers et permissions
- ▶ Concept de processus : programme en cours d'exécution
- ▶ Les fichiers ont une « place ».
- ▶ Les processus ont une « vie ».
- ▶ + un environnement de développement à la fois minimaliste et complet (compilateur C, shell, utilitaires, ...)

Bref historique

- 1964 Système d'exploitation Multics (*MULTiplexed Information and Computing Service*) développé par le MIT, les laboratoires Bell (AT&T) et General Electric
- 1969 Les laboratoires Bell quittent le projet Multics qui met trop longtemps à aboutir et Ken Thompson commence à travailler sur un nouveau système appelé Unics puis Unix.
- 1973 Unix est réécrit en langage C, inventé pour l'occasion par Dennis Ritchie et devient portable.
- 1974 L'université de Berkeley obtient un système Unix et commence à le modifier (début de BSD pour *Berkeley Software Distribution*).
- 1977 Steve Bourne (laboratoires Bell) crée le *Bourne shell* qui sera le shell par défaut de tous les Unix développés par les laboratoires Bell.

Bref historique (2)

- 1983 Unix System V est commercialisé par AT&T
- 1983 Richard Stallman lance le projet GNU (*GNU is Not Unix*) dont le but est d'écrire un système d'exploitation entièrement libre et compatible avec Unix.
- 1987 GNU C compiler (`gcc`, qui deviendra ensuite GNU Compiler Collection)
- 1989 Bourne again shell (`bash`) pour le projet GNU
- 1988 Débuts du standard POSIX pour uniformiser les nombreux Unix
- 1991 Linus Torvalds commence à travailler sur ce qui deviendra le noyau *Linux*.
- 1993 L'*Open Group*, en charge du standard POSIX acquiert les droits sur le nom Unix et définit ce qui est ou non un système Unix.

La famille des Unix

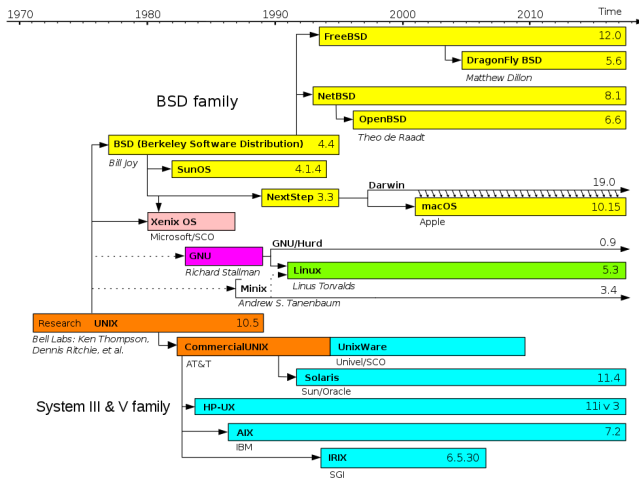


Figure – Guillem, Wereon, Hotmocha (copied from old version's history)Christoph S. (redrew the image with Inkscape), Public domain, via Wikimedia Commons

Unix aujourd'hui

GNU/Linux est un système Unix qui équipe la quasi-totalité des supercalculateurs, une part importante des serveurs et entre 1 et 2% des ordinateurs individuels.

FreeBSD est un système d'exploitation de type Unix, dont dérivent, par exemple, MacOS et le système d'exploitation des PlayStation 3 et 4 (5 ?)

Android équipe plus de 80% des smartphones et est basé sur le noyau Linux.

MacOS et iOS sont présents sur les ordinateurs personnels et les smartphones de la marque Apple.

Le standard POSIX

- ▶ Devant la multiplication des différents Unix, il était devenu nécessaire de *standardiser* les interfaces de ces différents systèmes d'exploitation pour pouvoir écrire des programmes (en C ou en shell par exemple) qui seraient *portables* sur les différents systèmes Unix.
- ▶ POSIX : *Portable Operating System Interface* + le X pour rappeler la filiation avec Unix.
- ▶ Standard public et mis à jour par un groupe totalement ouvert regroupant des industriels et des enthousiastes.
- ▶ La section *Shell and Utilities* du standard définit ce qu'est un shell Unix et décrit les commandes standard (comme `ls`, `cp`, etc).
- ▶ Le standard est plus ou moins bien respecté...
- ▶ Dernière version du standard : 2024 (je dois donc mettre à jour le cours!)

<https://pubs.opengroup.org/onlinepubs/9799919799/>

Organisation du cours

Le système Unix

Le shell

Demos 2025

Lignes de commandes

Interface

« Une interface est la couche limite par laquelle ont lieu les échanges et les interactions entre deux éléments. » (Wikipedia)

Interface physique :

- ▶ Téléscripneur, en anglais *teletypewriter* ou *teletype* (`tty` sous UNIX)
- ▶ Terminal vidéo (par exemple le VT100)
- ▶ Écran, clavier, souris
- ▶ Écran tactile

Interface logicielle :

- ▶ shell (interpréteur de commandes)
- ▶ interface graphique
- ▶ interface graphique des smartphones

Interface

« Une interface est la couche limite par laquelle ont lieu les échanges et les interactions entre deux éléments. » (Wikipedia)

Interface physique :

- ▶ Téléscripateur, en anglais *teletypewriter* ou *teletype* (`tty` sous UNIX)
- ▶ Terminal vidéo (par exemple le VT100)
- ▶ Écran, clavier, souris
- ▶ Écran tactile

Interface logicielle :

- ▶ shell (interpréteur de commandes)
- ▶ interface graphique
- ▶ interface graphique des smartphones

Le shell est une interface logicielle textuelle entre l'utilisateur et le noyau.

Le shell aujourd'hui

- ▶ *Bourne shell* (**sh**) de Steven Bourne (laboratoires Bell), shell par défaut d'Unix à partir de 1977 ; **aujourd'hui sh désigne tout shell qui a un comportement standard (ou presque).**
- ▶ *Korn shell* (**ksh**, **ksh88**, **ksh93**), de David Korn (laboratoires Bell) : s'inspire du *Bourne shell* et l'étend
- ▶ *C shell* (**csh**), de Bill Joy (université de Berkeley) et son descendant le **tcsh**, on oublie dans ce cours : trop différent
- ▶ *Bourne Again Shell* (**bash**) du projet GNU, shell par défaut dans la plupart des distributions Linux
- ▶ *Z shell* (**zsh**) de Paul Falstad (université de Princeton) qui reprend une partie de tous les shells précédemment cités, shell par défaut sur MacOS, très populaire
- ▶ *Debian Almquist Shell* (**dash**) : très proche du standard POSIX, minimaliste
- ▶ **yash**, **fish**, ...

Environnement Unix dans ce cours

- ▶ En mode interactif, on utilisera le shell **bash**.
- ▶ Mais les scripts respecteront le standard POSIX car :
 - ▶ plus petit (moins de choses à apprendre, donc on peut mieux les apprendre)
 - ▶ standard donc plus portable (devrait fonctionner avec tous les shells qui respectent le standard)
 - ▶ standard donc bien documenté

Environnement Unix dans ce cours

- ▶ En mode interactif, on utilisera le shell **bash**.
- ▶ Mais les scripts respecteront le standard POSIX car :
 - ▶ plus petit (moins de choses à apprendre, donc on peut mieux les apprendre)
 - ▶ standard donc plus portable (devrait fonctionner avec tous les shells qui respectent le standard)
 - ▶ standard donc bien documenté

Dans les salles de TP de l'institut Galilée :

- ▶ le système est une distribution GNU/Linux appelée *Debian*;
- ▶ l'environnement de bureau est *Gnome*;
- ▶ l'émulateur de terminal par défaut est *gnome-terminal*;
- ▶ le shell interactif par défaut est *bash*;
- ▶ le shell qui interprétera vos scripts est *dash*.

Organisation du cours

Le système Unix

Le shell

Demos 2025

Lignes de commandes

Demos 2025

- ▶ fichiers `a.out` inutiles : les trouver, les compter, les supprimer.
- ▶ un journal avec les 60 premiers jours
- ▶ insérer une ligne à des endroits spécifiques d'un gros projet

Organisation du cours

Le système Unix

Le shell

Demos 2025

Lignes de commandes

Commande

- ▶ Suite de mots qui se finit par un terminateur de commande (le plus souvent fin de ligne (*Line Feed*, '`\n`') avec la touche entrée)
- ▶ Le premier mot est le *nom de commande*.
- ▶ Les éventuels mots suivants sont les *arguments de la commande*.
- ▶ Les différents mots sont séparés par des *blancs* (espaces et/ou tabulations).
- ▶ Le nombre de blancs entre les mots n'a pas d'importance.
- ▶ Les blancs avant le nom de commande et avant le terminateur de commande n'ont pas d'importance.

Commande

- ▶ Suite de mots qui se finit par un terminateur de commande (le plus souvent fin de ligne (*Line Feed*, '`\n`') avec la touche entrée)
- ▶ Le premier mot est le *nom de commande*.
- ▶ Les éventuels mots suivants sont les *arguments de la commande*.
- ▶ Les différents mots sont séparés par des *blancs* (espaces et/ou tabulations).
- ▶ Le nombre de blancs entre les mots n'a pas d'importance.
- ▶ Les blancs avant le nom de commande et avant le terminateur de commande n'ont pas d'importance.

Exemple :

```
echo 1 deux trois 4
```

Commande

- ▶ Suite de mots qui se finit par un terminateur de commande (le plus souvent fin de ligne (*Line Feed*, '`\n`') avec la touche entrée)
- ▶ Le premier mot est le *nom de commande*.
- ▶ Les éventuels mots suivants sont les *arguments de la commande*.
- ▶ Les différents mots sont séparés par des *blancs* (espaces et/ou tabulations).
- ▶ Le nombre de blancs entre les mots n'a pas d'importance.
- ▶ Les blancs avant le nom de commande et avant le terminateur de commande n'ont pas d'importance.

Exemple :

```
echo 1 deux trois 4
```

Nom de commande : `echo`, 4 arguments.

Qu'est-ce qu'une commande ?

On distingue quatre types de noms commandes :

- ▶ les programmes externes, qui peuvent être des programmes compilés (par exemple depuis des fichiers sources C), ou interprétés (par exemple par python, perl, etc, ou le shell lui-même!) ;
- ▶ les primitives du shell, c'est-à-dire des fonctionnalités internes au shell ;
- ▶ les fonctions shell, que l'on abordera plus tard si on a le temps ;
- ▶ les alias, qui sont des raccourcis pour d'autres commandes.

type type

La commande `type` permet de connaître le type d'un nom de commande :

```
$ type who date echo cd ls mkcd type
who est /usr/bin/who
date est /bin/date
echo est une primitive du shell
cd est une primitive du shell
ls est un alias vers "ls --color=auto"
mkcd est une fonction
mkcd ()
{
    mkdir "$1";
    cd "$1"
}
type est une primitive du shell
```


man man

Obtenir de l'aide :

- ▶ pour un programme externe : `man`
- ▶ pour une primitive du shell, dans `bash`, `help`
- ▶ pour de nombreuses commandes (non standard), option `--help`

```
$ type type
```

```
type est une primitive du shell
```

```
$ help type
```

```
type: type [-afptP] nom [nom ...]
```

```
Affiche des informations sur le type de commande.
```

Pour chaque NOM, indique comment il serait
interprete s il etait utilise comme un nom de
commande.

...

```
$ type id
```

```
id est /usr/bin/id
```

```
$ man id
```

```
$ type man
```

```
man est /usr/bin/man
```

```
$ man man
```